



دانشگاه صنعتی شریف

بازتولید مقاله

Dynamics of Sequential Decision Making^[1]

(دینامیک تصمیم‌گیری گام‌به‌گام)

مهدی ابوالحسنی - ۴۰۲۱۱۱۲۱۵

علیرضا حبیب زاده - ۹۹۱۰۹۳۹۳

علوم اعصاب - دکتر سامان مقیمی

زمستان ۱۴۰۳

چکیده

این مقاله یک مدل جدید برای درک فرایند تصمیم‌گیری در سیستم‌هایی با رفتارهای پیچیده و دینامیکی، مانند حیوانات یا دستگاه‌های هوشمند معرفی می‌کند. این مطالعه مدلی دینامیکی برای Cognitive-State Machines (CSMs) ارائه می‌دهد که از طریق مجموعه‌ای از معادلات دیفرانسیل معمولی (ODEs) با نقاط تصمیم‌گیری گسسته توصیف می‌شوند. این مدل محرک‌های محیطی و معیارهای تصمیم‌گیری را در نظر می‌گیرد و چهارچوب پیشنهادی بر اصل Winnerless Competition (WLC) تعمیم‌یافته تکیه دارد که در آن دینامیک گذر بین حالت‌های شناختی رقیب، تصمیم‌های پیچیده را تعیین می‌کند. نویسندگان دو استراتژی متضاد، یعنی high-risk و risk-aversion، را مقایسه کرده و اثربخشی آنها را در طولانی کردن دنباله رفتارها (یا «عمر») تحلیل می‌کنند که در ادامه به توضیح آن‌ها می‌پردازیم.

۱ معادلات اصلی

دینامیک سیستم با دو معادله اصلی مدل می‌شود. معادله اول رقابت بین حالت‌های شناختی $a_i(t)$ را مدل می‌کند که شبیه به معادلات شکار و شکارچی است:

$$\dot{a}_i = a_i \left[\sigma_i(I, t) - \left(a_i + \sum_{j \neq i}^N \rho_{ij} a_j \right) \right] + \eta_i(t) \quad (1)$$

که در آن $a_i(t)$ نشان‌دهنده فعالیت حالت شناختی i -ام، ρ_{ij} شدت تعامل بین حالت‌ها، $\eta_i(t)$ ترم نوفه، و N تعداد حالت‌های شناختی است. معادله دوم تکامل پارامترهای کنترل $\sigma_i(t, I)$ را توصیف می‌کند:

$$\tau \dot{\sigma}_i = - \frac{\partial U_i(\sigma_i, I)}{\partial \sigma_i} \quad (2)$$

در اینجا $U_i(\sigma_i, I)$ یک تابع پتانسیل با مینیمم‌های گسسته است که تحت تأثیر محرک‌ها I قرار دارد.

مدل برای ایجاد دنباله‌های گذرای حالت‌ها از Stable Heteroclinic Sequences (SHS) استفاده می‌کند که انتقال‌ها بین نقاط زینی پایدار S_i در سیستم را توصیف می‌کنند. پایداری با برآورده شدن شرایط زیر تضمین می‌شود:

$$1 - \rho_{i_k-1, i_k} \cdot \rho_{i_k, i_k-1} \neq 0, \quad (4)$$

$$-\bar{\sigma}_{i_k} < \bar{\sigma}_{i_{k-1}} - \rho_{i_{k-1}i_k} \bar{\sigma}_{i_k}, \quad (5)$$

$$\bar{\sigma}_i - \rho_{ii_k} \bar{\sigma}_{i_k} < \bar{\sigma}_{i_{k-1}} - \rho_{i_{k-1}i_k} \bar{\sigma}_{i_k}, \quad i \notin \{i_{k-1}, i_k, i_{k+1}\}. \quad (6)$$

دو استراتژی تصمیم‌گیری در این مدل مورد بررسی قرار گرفته است. در high-risk strategy، سیستم با اولویت دادن به انتقالات سریع، زمان صرف‌شده در نقاط زینی را به حداقل می‌رساند و بزرگ‌ترین مقادیر ویژه مثبت را انتخاب می‌کند:

$$0 < \lambda_{j_0(q)i_1} < \lambda_{j_0(q_0)i_1}, \quad q \neq q_0. \quad (7)$$

این رویکرد اغلب دنباله‌های طولانی‌تری ایجاد می‌کند، اما تغییرپذیری و احتمال «گذار فازی» در رفتار را نیز افزایش می‌دهد. در مقابل، risk-aversion strategy با انتخاب نقاط زینی با بالاترین شاخص پایداری ($\nu_i > 1$) بر پایداری تمرکز دارد:

$$1 < \nu_{i_1}^q < \nu_{i_1}^{q_0}, \quad q \neq q_0. \quad (8)$$

این استراتژی تغییرپذیری را محدود کرده و از رفتار سرگردان اجتناب می‌کند، اما دنباله‌های کوتاه‌تری به همراه دارد. (منظور از دنباله یک زنجیره تصمیم‌گیری است. برای درک بهتر به تصویر ۱ مقاله اصلی مراجعه کنید. [۱])

۲ یافته‌ها

در high-risk strategy، دنباله‌های رفتاری طولانی‌تر و غنی‌تری تولید می‌شود. همچنین گذارهای فازی مشاهده می‌شود که در آن سیستم یا به نقاط ثابت پایدار می‌رسد یا وارد رفتارهای تکراری یا چرخه محدود می‌شود. افزایش پیچیدگی تصمیم‌گیری (M) و پیچیدگی حالت شناختی (N) به دنباله‌های طولانی‌تر منجر می‌شود. در مقابل، risk-aversion strategy دنباله‌های کوتاه‌تری ایجاد می‌کند که با پیچیدگی (N) به صورت نمایی کاهش می‌یابند و مستقل از تعداد انتخاب‌ها (M) هستند. این استراتژی رفتار پایدار و قابل تکراری ارائه می‌دهد و حساسیت کمتری به نوفه محیطی نشان می‌دهد.

۳ پیامدها و کاربردها

مدل پیشنهادی شکاف بین reproducible decision-making یا تصمیم‌گیری قابل بازتولید و حساسیت به تغییرات محیطی را پر می‌کند. این مدل بینش‌هایی درباره رفتار حیوانات (توضیح چگونگی ظهور استراتژی‌های بقا مانند risk-taking یا ریسک کردن در مقابل risk-aversion یا گریز از ریسک)، هوش مصنوعی (ارائه چهارچوبی قوی برای طراحی سیستم‌هایی که به محیط تطبیق پیدا می‌کنند و به صورت گام به گام تصمیم می‌گیرند)، و علوم اعصاب (توضیح یافته‌های تجربی نظیر واکنش‌های سریع‌تر میمون‌های macaque برای پاداش‌های بزرگ‌تر) ارائه می‌دهد. نهایتاً با شبیه‌سازی حالت‌های شناختی رقیب و دینامیک گذرای آنها، این مقاله پایه‌ای ریاضی برای درک رفتارهای هوشمند پیاپی در شرایط عدم قطعیت ارائه می‌کند.

۴ بازپاده‌سازی

این کد شبیه‌سازی دینامیک‌های تصمیم‌گیری (DM) با ریسک بالا را در یک ماشین حالت شناختی (CSM) بر اساس مدل رقابت Lotka-Volterra یا شکار و شکارچی پاده‌سازی می‌کند. تمرکز بر روی نمایش دینامیک‌های گذرا در دو سناریو است:

آ) رفتار معمولی DM که در آن حالت‌ها به‌طور غیرقابل پیش‌بینی تکامل می‌یابند.

ب) الگوهای تکراری DM که توسط یک محیط ساختاریافته و تکراری معرفی می‌شوند.

معادلات مدل تعاملات رقابتی بین حالت‌های شناختی (a_i) را توصیف می‌کنند که تحت تأثیر پتانسیل‌ها (σ_i) و شدت تعاملات (ρ_{ij}) قرار دارند و همچنین نوفه گاوسی به این دینامیک‌ها اضافه شده.

```

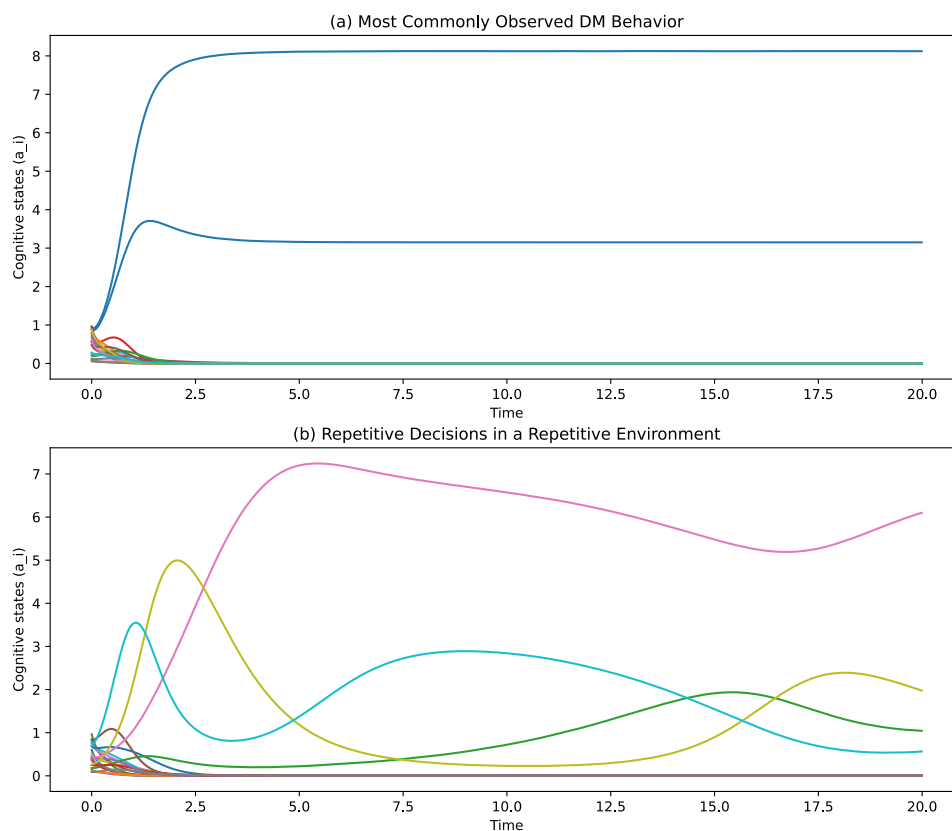
1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(99109393 + 402111215)
4
5 # Parameters
6 N = 20 # Number of cognitive states
7 M = 5 # Number of possible choices
8 T = 20 # Total time for simulation
9 dt = 0.1 # Time step
10
11 # Function to simulate high-risk DM dynamics
12 def simulate_high_risk_dm(N, M, T, dt, repetitive=False):
13     time = np.arange(0, T, dt)
14     a = np.zeros((N, len(time))) # Cognitive states over time
15
16     # Initialize parameters
17     if repetitive:
18         sigma = np.linspace(5, 10, N) # Create repetitive environment with fixed potentials
19     else:
20         sigma = np.random.uniform(5, 10, N) # Cognitive state potentials
21     rho = np.zeros((N, N)) # Competition matrix
22     for i in range(N):
23         for j in range(N):
24             if i != j:
25                 rho[i, j] = np.random.uniform(5.0, 5.1) # Interaction strengths
26
27     # Noise term
28     eta = lambda: np.random.normal(0, 0.1, N) # Small Gaussian noise
29
30     # Initial conditions
31     a[:, 0] = np.random.uniform(0, 1, N)
32
33     # Dynamics: Lotka-Volterra type competition
34     for t in range(1, len(time)):
35         da = np.zeros(N)
36         for i in range(N):
37             competitive_term = sum(rho[i, j] * a[j, t - 1] for j in range(N) if j != i)
38             da[i] = a[i, t - 1] * (sigma[i] - a[i, t - 1] - competitive_term) + eta()[i]
39
40     # Update cognitive states
41     a[:, t] = a[:, t - 1] + da * dt
42     a[:, t] = np.maximum(0, a[:, t]) # Ensure states remain non-negative
43
44     return time, a
45
46 # Simulate for (a) Most commonly observed DM behavior
47 time_a, a_a = simulate_high_risk_dm(N, M, T, dt, repetitive=False)
48

```

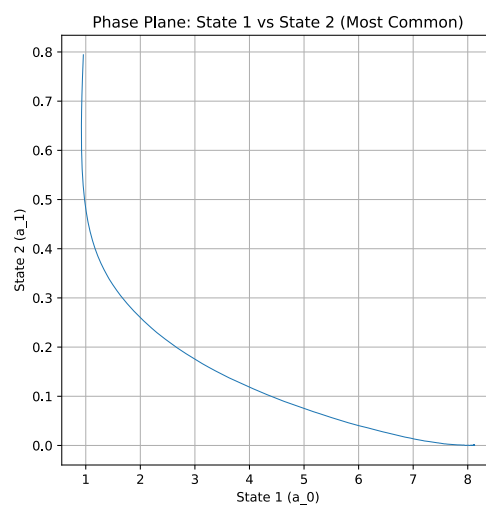
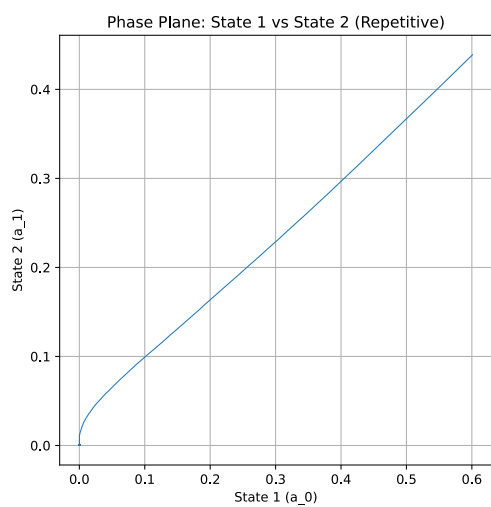
```

۴۹ # Simulate for (b) Repetitive decisions in a repetitive environment
۵۰ time_b, a_b = simulate_high_risk_dm(N, M, T, dt, repetitive=True)
۵۱
۵۲
۵۳ plt.figure(figsize=(12, 10))
۵۴ # (a) Most commonly observed DM behavior
۵۵ plt.subplot(2, 1, 1)
۵۶ plt.plot(time_a, a_a.T)
۵۷ plt.xlabel("Time")
۵۸ plt.ylabel("Cognitive states (a_i)")
۵۹ plt.title("(a) Most Commonly Observed DM Behavior")
۶۰
۶۱ # (b) Repetitive decisions in a repetitive environment
۶۲ plt.subplot(2, 1, 2)
۶۳ plt.plot(time_b, a_b.T)
۶۴ plt.xlabel("Time")
۶۵ plt.ylabel("Cognitive states (a_i)")
۶۶ plt.title("(b) Repetitive Decisions in a Repetitive Environment")
۶۷ plt.savefig('plot1.svg')
۶۸ plt.show()
۶۹
۷۰ # Phase plane plotting function
۷۱ def plot_phase_plane(a, state1, state2, title):
۷۲     plt.figure(figsize=(6, 6))
۷۳     plt.plot(a[state1, :], a[state2, :], lw=8.0)
۷۴     plt.xlabel(f"State {state1 + 1} (a_{state1})")
۷۵     plt.ylabel(f"State {state2 + 1} (a_{state2})")
۷۶     plt.title(title)
۷۷     plt.grid()
۷۸     plt.savefig(f'{title}.svg')
۷۹     plt.show()
۸۰
۸۱
۸۲ plot_phase_plane(a_a, 0, 1, "Phase Plane: State 1 vs State 2 (Most Common)")
۸۳ plot_phase_plane(a_b, 0, 1, "Phase Plane: State 1 vs State 2 (Repetitive)")

```



شکل ۱: نمودار زمانی دینامیک‌های تصمیم‌گیری با ریسک بالا.



شکل ۲: نمودار فاز: نمودارهای فاز مربوط به مسیرهای تعامل بین دو حالت شناختی انتخاب‌شده نشان داده شده‌اند. این مسیرها دینامیک‌های گذرا و رقابت متقابل بین حالت‌ها را تحت تصمیم‌گیری با ریسک بالا نشان می‌دهند.

۱.۴ گذر فاز

این شبیه‌سازی از مدل دینامیک Lotka-Volterra برای توصیف رقابت گذرای بین حالت‌های شناختی (a_i) استفاده می‌کند که تحت تأثیر پارامترهای داخلی (σ_i)، شدت تعاملات (ρ_{ij}) و نوفه گاوسی (η) قرار دارند. سیستم تا زمانی تکامل می‌یابد که یک شرط خاتمه (فعالیت کلی پایین) برآورده شود. نتایج پس از چندین بار اجرا جمع‌آوری شده و میانه‌های L و C محاسبه می‌شوند.

نتایج نشان می‌دهند:

آ) گذارهای فاز در سیستم‌های بزرگ‌تر ($N = 25, 50$) که در آن دنباله‌های طولانی‌تر با افزایش M ظاهر می‌شوند.

ب) رفتار پایدار در سیستم‌های کوچک‌تر ($N = 10$).

```

۱ num_simulations = 100 # Number of simulations per combination
۲ M_values = np.arange(2, 23, 1) # Number of choices
۳ N_values = [10, 25, 50] # Number of cognitive states
۴ noise_level = 0.1 # Noise amplitude
۵
۶ def simulate_csm(N, M, noise_level, max_time_steps=500, dt=0.1):
۷     # Initialize cognitive states and parameters
۸     a = np.random.uniform(1.0, 0.1, N) # Initial cognitive states
۹     sigma = np.random.uniform(5, 10, N) # Internal parameters
۱۰    rho = np.zeros((N, N)) # Interaction matrix
۱۱
۱۲    for i in range(N):
۱۳        for j in range(N):
۱۴            if i != j:
۱۵                rho[i, j] = sigma[j] / sigma[i] + 5.0 if j == i + 1 else 0.0
۱۶
۱۷    life_length = 0
۱۸    nodes_involved = set()
۱۹
۲۰    for t in range(max_time_steps):
۲۱        eta = np.random.normal(0, noise_level, N) # Noise
۲۲
۲۳        # Update cognitive states (Lotka-Volterra dynamics)
۲۴        da = a * (sigma - (a + rho @ a)) + eta
۲۵        a += da * dt
۲۶        a = np.clip(a, 0, np.inf) # Ensure non-negative values
۲۷
۲۸        # Decision-making rule: move to the fastest-growing state
۲۹        active_state = np.argmax(a)
۳۰        nodes_involved.add(active_state)
۳۱        life_length += 1
۳۲
۳۳        # Check termination condition (end of life)
۳۴        if life_length > 1 and np.sum(a) < 1e-3:
۳۵            break
۳۶
۳۷    return life_length, len(nodes_involved)
۳۸

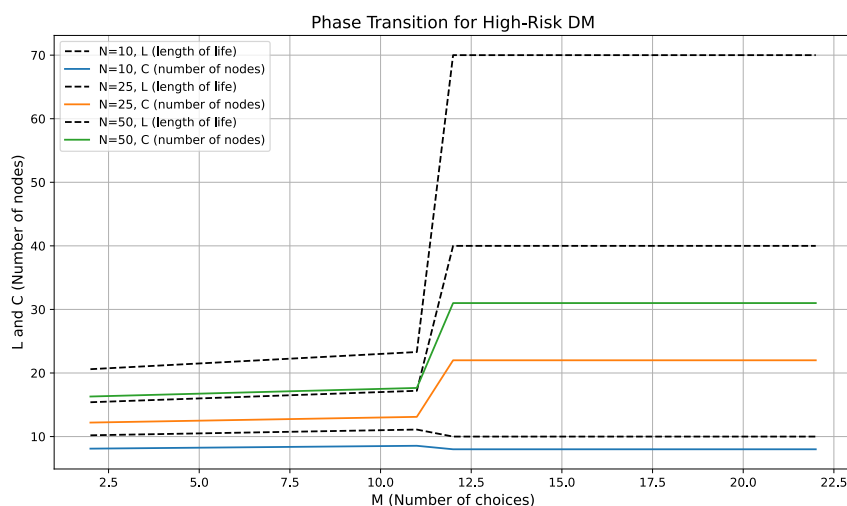
```

```

۳۹ L_medians = {N: [] for N in N_values}
۴۰ C_medians = {N: [] for N in N_values}
۴۱
۴۲ for N in N_values:
۴۳     for M in M_values:
۴۴         life_lengths = []
۴۵         node_counts = []
۴۶         for _ in range(num_simulations):
۴۷             life_length, nodes_involved = simulate_csm(N, M, noise_level)
۴۸             life_lengths.append(life_length)
۴۹             node_counts.append(nodes_involved)
۵۰         L_medians[N].append(np.median(life_lengths))
۵۱         C_medians[N].append(np.median(node_counts))
۵۲
۵۳ plt.figure(figsize=(10, 6))
۵۴
۵۵ for N in N_values:
۵۶     plt.plot(M_values, L_medians[N], 'k--', label=f'N={N}, L (length of life)', linewidth=5.1)
۵۷     plt.plot(M_values, C_medians[N], label=f'N={N}, C (number of nodes)', linewidth=5.1)
۵۸
۵۹ plt.xlabel('M (Number of choices)', fontsize=12)
۶۰ plt.ylabel('L and C (Number of nodes)', fontsize=12)
۶۱ plt.title('Phase Transition for High-Risk DM', fontsize=14)
۶۲ plt.legend()
۶۳ plt.grid(True)
۶۴ plt.savefig('plt4.svg')
۶۵ plt.show()

```

نتیجه



شکل ۳: گذار فاز برای تصمیم‌گیری با ریسک بالا. میان‌ه طول عمر L (خط چین) و میان‌ه تعداد گره‌های درگیر در دنباله (خط‌های پیوسته) در مقابل تعداد انتخاب‌ها M برای ماشین حالت شناختی (CSM) با تعداد حالت‌های شناختی $N = 10, 25, 50$.

۲.۴ توزیع احتمال

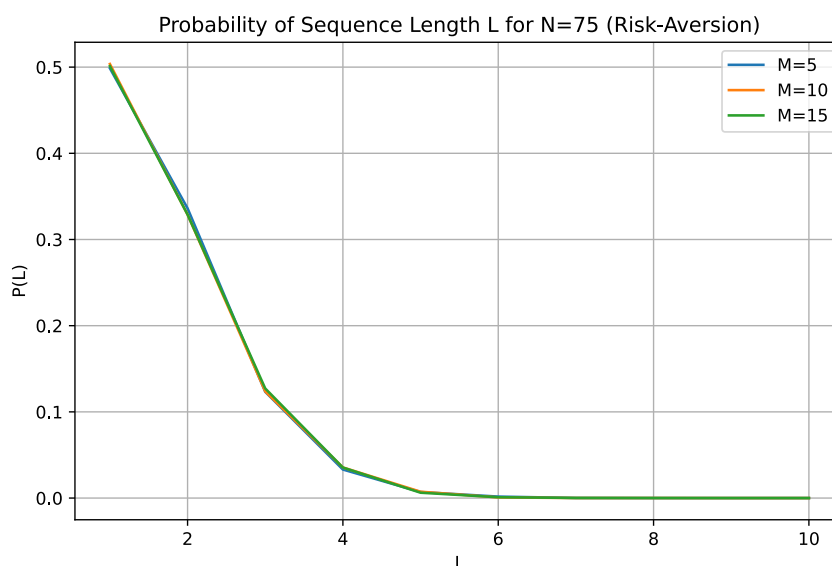
این کد توزیع احتمالی $P(L)$ را برای طول دنباله‌ها L در سیستمی با اندازه $N = 75$ تحت استراتژی تصمیم‌گیری اجتناب از ریسک شبیه‌سازی می‌کند. در این شبیه‌سازی از یک قانون استفاده می‌شود که در آن طول دنباله افزایش می‌یابد مادامی که مقدار تصادفی از $(L/(L+1))$ بیشتر باشد، که نشان‌دهنده تصمیم‌گیری محتاطانه است. این فرآیند برای هر تعداد انتخاب ($M = 5, 10, 15$) به تعداد 20,000 بار تکرار می‌شود تا احتمالات محاسبه شوند و سپس نرمال‌سازی صورت می‌گیرد.

نمودار حاصل نشان می‌دهد چگونه احتمال دستیابی به طول دنباله L با M تغییر می‌کند و تأثیر پیچیدگی انتخاب‌ها را بر رفتار اجتناب از ریسک نشان می‌دهد.

```

1 # Parameters
2 N = 75 # System size
3 M_values = [5, 10, 15] # Number of choices
4 num_runs = 20000 # Number of simulations
5
6 # Simulate risk-aversion decision-making
7 def simulate_risk_aversion(N, M, num_runs):
8     probabilities = np.zeros(N)
9
10    for _ in range(num_runs):
11        L = 1 # Start sequence length
12        while np.random.rand() > (L / (L + 1)) and L < N: # Risk-aversion rule
13            L += 1
14        probabilities[L - 1] += 1
15
16    probabilities /= num_runs # Normalize to probabilities
17    return probabilities
18
19 # Run simulations and store results
20 results = {}
21 for M in M_values:
22     results[M] = simulate_risk_aversion(N, M, num_runs)
23
24 # Plot results
25 plt.figure(figsize=(8, 5))
26 for M in M_values:
27     plt.plot(range(1, 10 + 1), results[M][:10], label=f"M={M}")
28
29 plt.xlabel("L")
30 plt.ylabel("P(L)")
31 plt.title("Probability of Sequence Length L for N=75 (Risk-Aversion)")
32 plt.legend()
33 plt.grid()
34 plt.savefig('plot5.svg')
35 plt.show()

```



شکل ۴: توزیع احتمالی $P(L)$ برای طول دنباله‌ها L در استراتژی تصمیم‌گیری اجتناب از ریسک با $N = 75$ و تعداد انتخاب‌های متغیر $(M = 5, 10, 15)$.

۵ نتیجه‌گیری

این مقاله یک چهارچوب یا راه‌حل دینامیکی برای تصمیم‌گیری پیاپی بر اساس رقابت بین حالت‌های شناختی معرفی می‌کند و استراتژی‌های «ریسک بالا» و «اجتناب از ریسک» را مقایسه می‌کند. شبیه‌سازی‌ها نشان می‌دهند که استراتژی‌های ریسک بالا منجر به دنباله‌های طولانی‌تر می‌شوند و گذارهای فاز را نشان می‌دهند، در حالی که استراتژی‌های اجتناب از ریسک دنباله‌های کوتاه‌تر و پایداری تولید می‌کنند که احتمالات آنها به صورت نمایی کاهش می‌یابند.

شبیه‌سازی‌ها همچنین نشان می‌دهند که چگونه پیچیدگی سیستم و تعداد انتخاب‌ها بر رفتار تصمیم‌گیری تأثیر می‌گذارند و توانایی مدل ساده مقاله را در توصیف دینامیک‌های متنوع تصمیم‌گیری تأیید می‌کنند. شبیه‌سازی این سیستم‌ها همچنین اهمیت پارامترهایی مانند نوبه و شدت تعاملات بین حالت‌ها را در بازتولید رفتارها نشان می‌دهد.

مراجع

- [1] Mikhail I. Rabinovich, Ramón Huerta, and Valentin Afraimovich. Dynamics of sequential decision making. *Phys. Rev. Lett.*, 97:188103, Nov 2006.